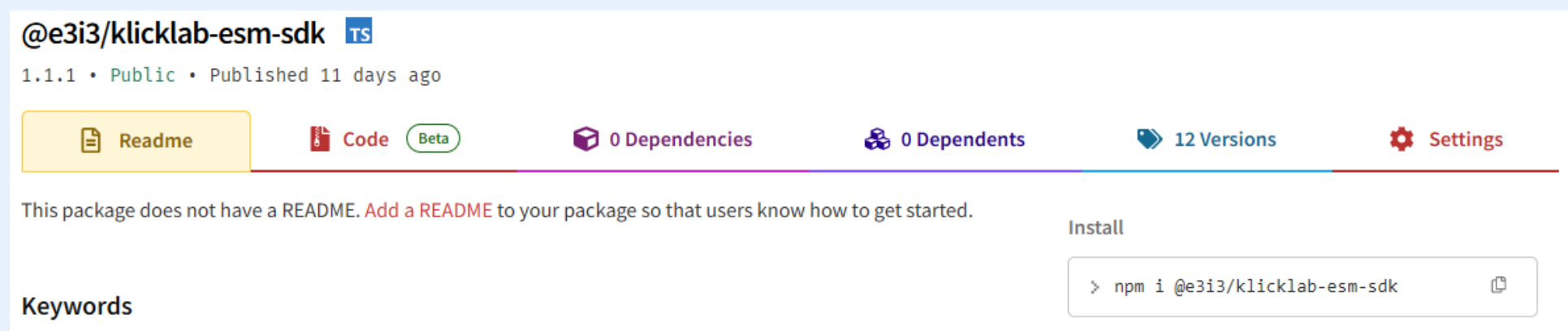


Introduction



KlickLab은 웹사이트에서 발생하는 행동 데이터를 **실시간 수집·구조화**하여 운영자가 **Data-Driven** 방식으로 합리적인 의사결정을 내릴 수 있도록 지원하는 **클릭스트림 분석 플랫폼**입니다.

SDK & TrafficGenerator



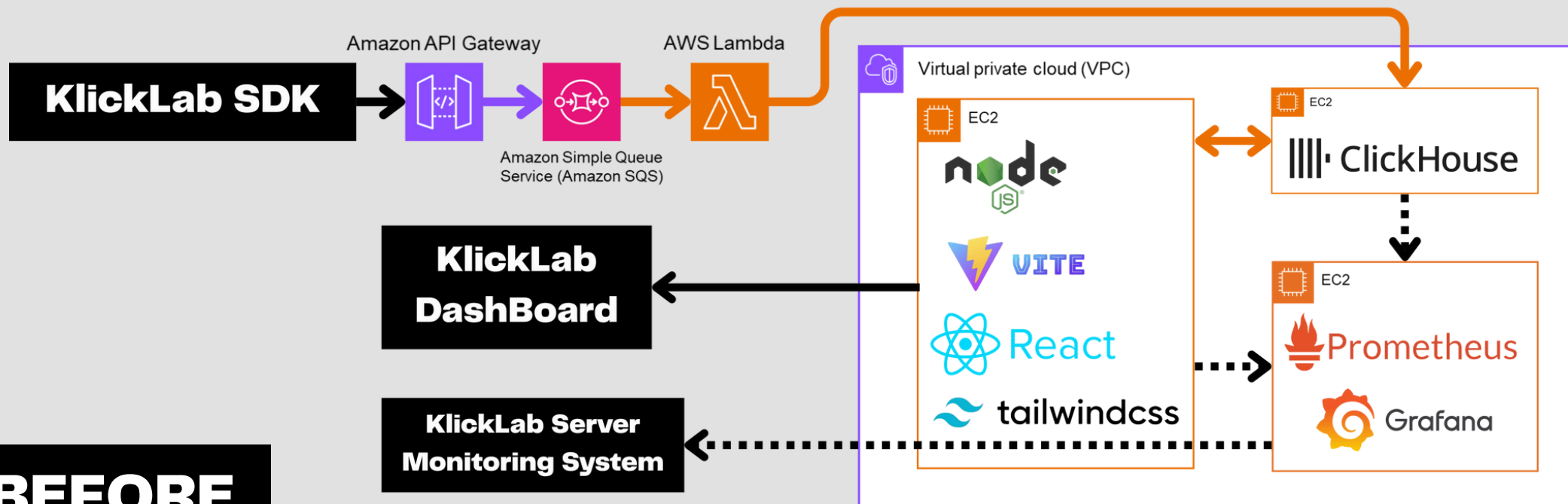
- 클릭·스크롤·페이지뷰·폼 전송까지 행동 로그를 이벤트 위임으로 감지
- 모든 클릭 이벤트는 **단일 document** 리스너로 처리
- UMD**(AMD·CommonJS·브라우저) 호환으로 모든 환경에 즉시 적용

2025-07-18 23:15:41 [Progress] Success: 1064015, Fail: 3319, RPS: 4924.01
2025-07-18 23:15:42 [Progress] Success: 1069112, Fail: 3319, RPS: 5096.37
2025-07-18 23:15:43 [Progress] Success: 1074460, Fail: 3319, RPS: 5347.07
2025-07-18 23:15:44 [Progress] Success: 1079289, Fail: 3319, RPS: 4826.78

- Go 언어** 기반으로 제작, 초당 수만 건의 요청 생성 가능
- 워커 수, 요청 간 간격, 지속 시간 등을 유연하게 설정 가능
- GC 멈춤 현상과 인프라 수용 **한계 검증에 활용**된 핵심 도구

Architecture

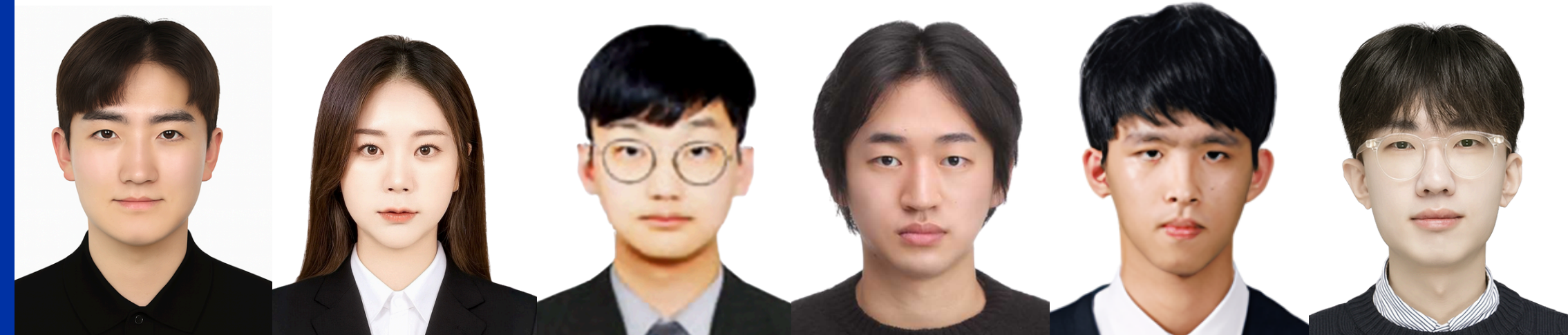
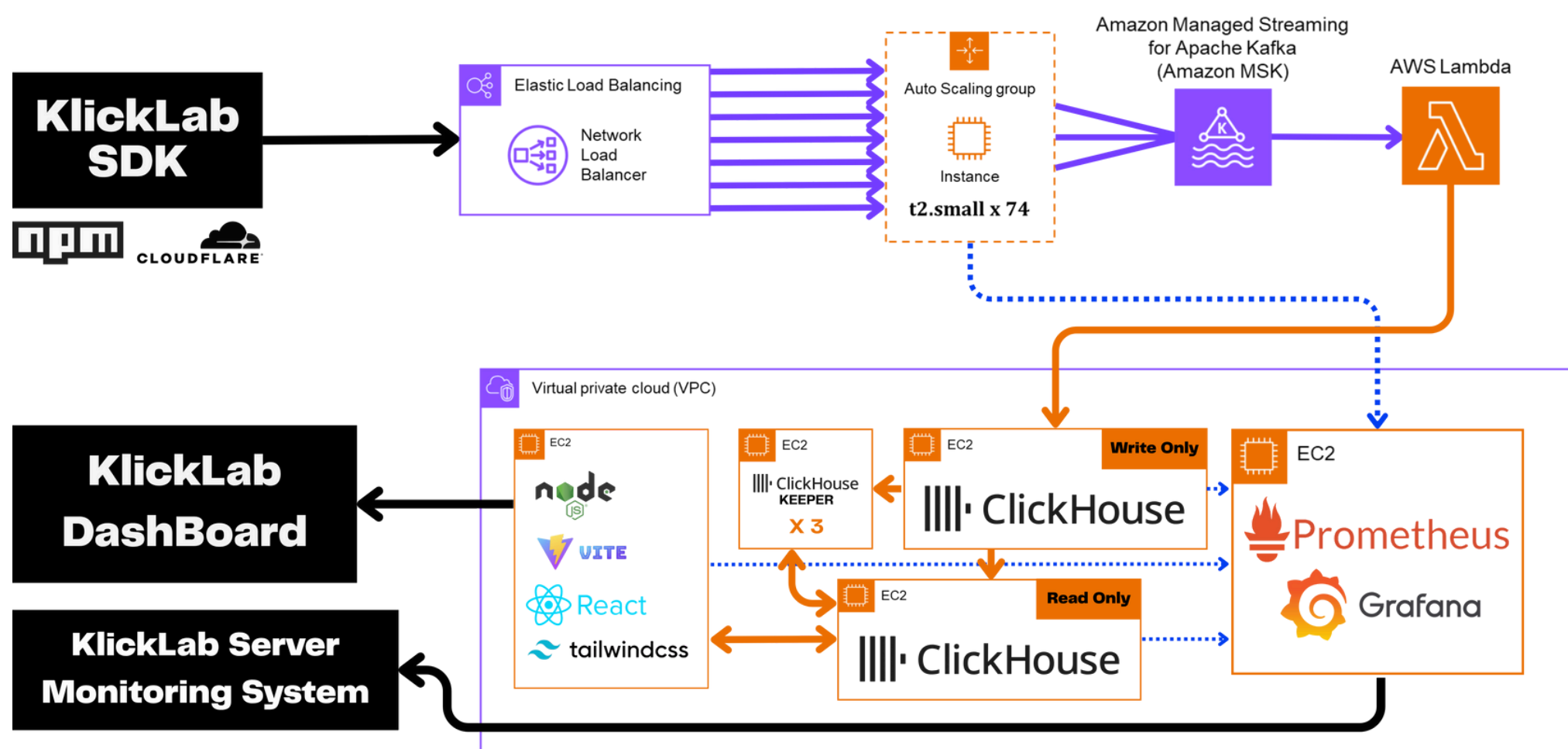
- API Gateway의 429 스로틀링·L7 오버헤드로 유실·지연이 발생
- 순서/리플레이 불가능한 SQS 사용으로 순서 보장·데이터 재처리 불가능



BEFORE

AFTER

- L4 NLB+오토스케일링으로 전환해서 10k RPS에서도 100% 수신율 확보
- MSK(Kafka)로 전환해 순서 보장·데이터 재처리 가능한 파이프라인 구축
- ClickHouse Keeper 기반 복제를 통해 고가용성 및 확장성 강화



김관수

김재현

박준식

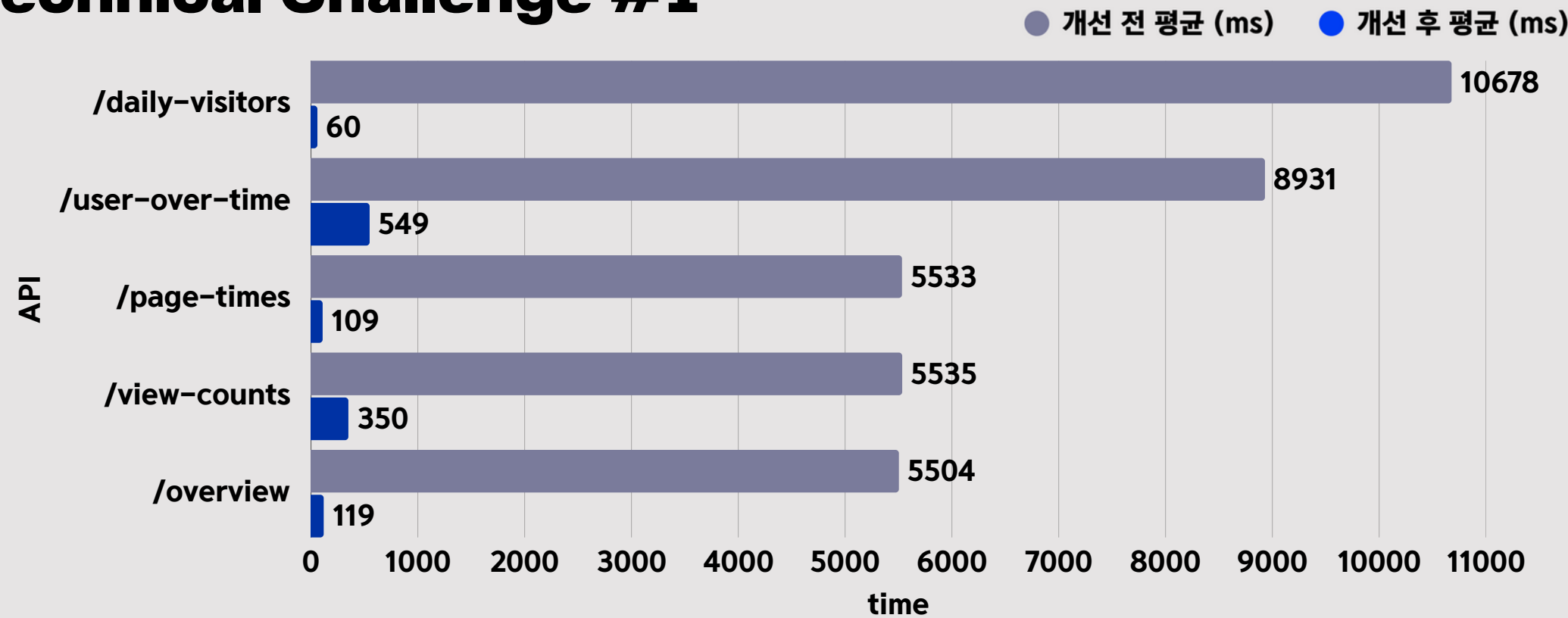
오주영

윤석주

이현재

수집-조회를 위한 Dual-Track

Technical Challenge #1



BEFORE

- 모든 쿼리가 **대용량 Raw Table**에 직접 접근
- 쓰기/읽기 작업이 **단일 DB**에 집중
- 그 결과 **응답 지연**과 **리소스 병목** 발생

AFTER

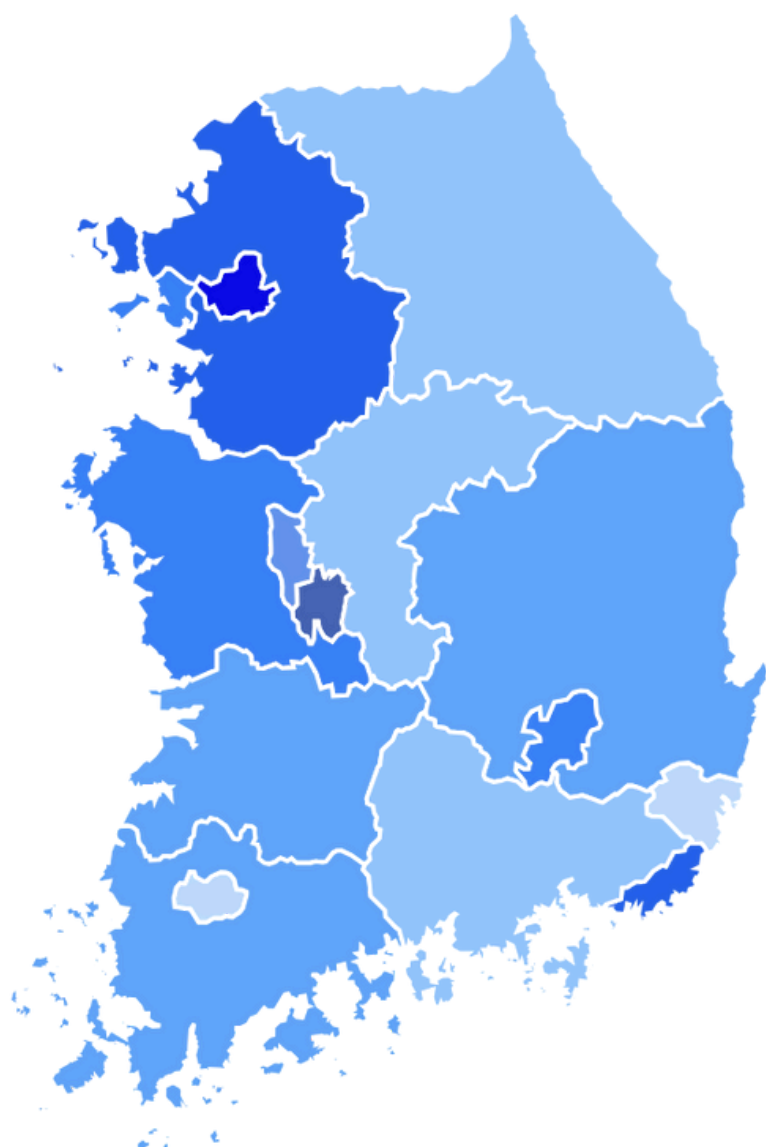
- Aggregation Table** 도입으로 쿼리 속도 개선
- Materialized View** 도입으로 Near-Real-time 분석 가능
- 쓰기 / 읽기 DB** 분리로 **Race Condition 해소**
- 1,000만 건 데이터 기준 **응답 시간 최대 99.43% 단축**

비정형 데이터 시각화 전략

Technical Challenge #2

지역별 활성 사용자 Heat Map

- 외부 라이브러리 비의존 방식으로 구현
- SVG 벡터 기반으로 사용자 정의 지도 형상 구성
- 지역별 사용자 분포 데이터와 매핑
- 각 지역 영역에 대한 정교한 인터랙션 기능 구현
- 반응형 및 동적 스타일 적용



사용자 여정 Sankey Chart

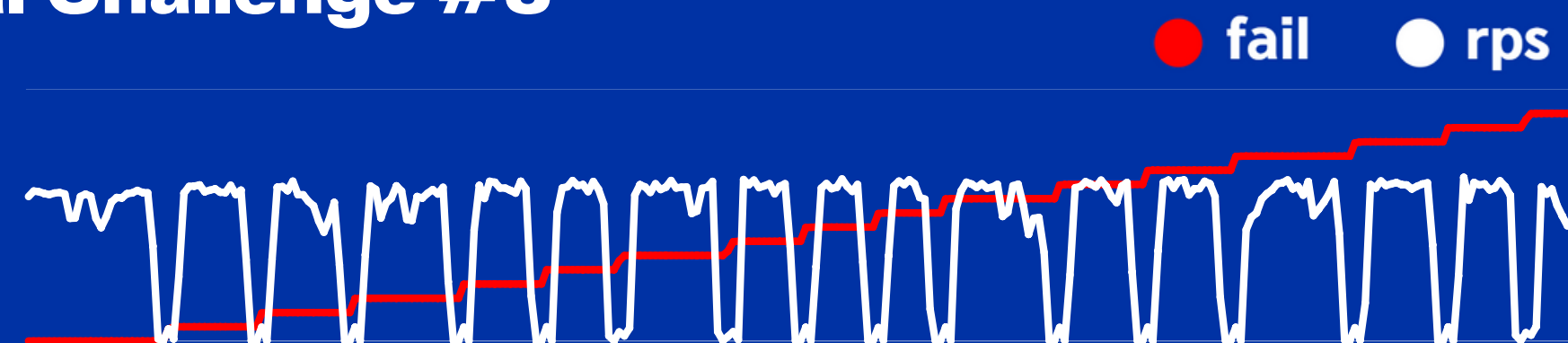
- 계층적 분기형 사용자 여정 데이터 기반 구조 설계
- 노드 기반 고복잡도 트리 형태 대응
- 복잡한 사용자 흐름 데이터의 효율적 구조화
- 실시간 대응용 동적 렌더링 기법 적용

성능과 비용, 두 마리 토끼 잡기

Technical Challenge #3

BEFORE

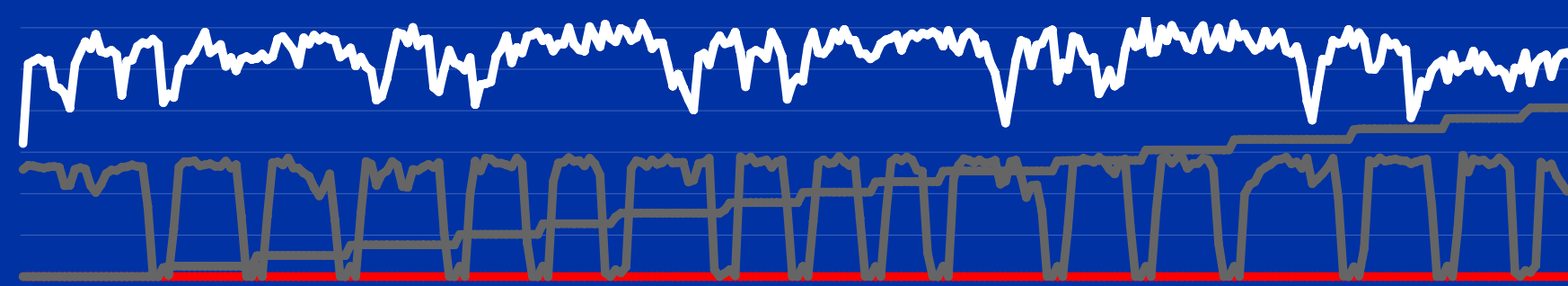
RPS 3,938



- c6in.xlarge 인스턴스 17대를 구성했지만 **GC로 인한 Stop-the-World**
- Node.js의 단일 스레드 구조로 인해 약 **30초 주기로 RPS가 급격히 하락**
- 고사양 인스턴스를 늘릴수록 비용이 급증하고, vCPU 할당량 한계에 부딪힘

AFTER

RPS 10,682



- t2.small 인스턴스 74대를 활용하여 처리량을 인스턴스별로 분산
- GC는 여전히 발생하지만, 전체가 아닌 개별 인스턴스 단위로만 영향을 받음
- 평균 RPS가 3,938에서 10,682로 증가함과 동시에 비용 56% 절감**